

RandomFourierFeatures_Volterra_LogSfBM

Random Fourier Features for the SfBM Kernel and Stochastic Volterra Equations

Technical Documentation — `RFFVolterraSfBM_Class.py` &
`RFFVolterraSfBM_Uutils.py`

Othmane Zarhali

June 22, 2026

Abstract

This document is a complete mathematical and software reference for `RFFVolterraSfBM_Class.py` and `RFFVolterraSfBM_Uutils.py`, which implement the **Random Fourier Feature (RFF)** approximation framework for the Stationary fractional Brownian Motion (SfBM) kernel and its application to simulating Stochastic Volterra Equations (SVEs). Three classes are provided: `SpectralDensity`, which evaluates the spectral density f_ω of the SfBM kernel via a radial numerical integral or a closed-form hypergeometric expression; `SpectralSampler`, which draws M spectral frequencies η_m from f_ω using Hamiltonian Monte Carlo or Acceptance-Rejection and reconstructs the kernel via the cosine characteristic function $K^M(u) = \frac{\nu^2}{2} \mathbb{E}[\cos(\eta^\top u)]$; and `SVE_Simulator`, which integrates a one-dimensional SVE using either the exact $\mathcal{O}(N^2)$ Euler scheme or the fast $\mathcal{O}(NM)$ Euler-with-RFF scheme, and provides Monte-Carlo routines for strong L^p error and weak observable error as functions of M .

Contents

1	Mathematical Background	3
1.1	The SfBM kernel	3
1.2	Spectral density f_ω and the Bochner identity	3
1.3	Random Fourier Features (RFF)	4
1.4	Stochastic Volterra Equation (SVE)	4
1.4.1	Algorithm 1: Euler scheme ($\mathcal{O}(N^2)$)	4
1.4.2	Algorithm 2: Euler with RFF ($\mathcal{O}(NM)$)	4
2	Module Architecture	6
3	Class <code>SpectralDensity</code>	7
3.1	Constructor: <code>__init__(nu2, H, T, d=1)</code>	7
3.2	<code>spectral_density_numeric(x, method='simpson')</code>	7
3.3	<code>spectral_density_closedform(x)</code>	7
3.4	<code>compare(xmin, xmax, npts)</code>	7
3.5	<code>save_comparison(path, xmin=1.0, xmax=8.0, npts=30)</code>	7
3.6	<code>inverse_fourier_transform(x)</code>	8
4	Class <code>SpectralSampler</code>	9
4.1	Constructor: <code>__init__(nu2, H, T, d=1, n_samples=1000, method='Hamiltonian MC')</code>	9

4.2	Radial density <code>_radial_pdf(r)</code>	9
4.3	Sampling backend 1: <code>_sample_hmc()</code> — Hamiltonian Monte Carlo	9
4.4	Sampling backend 2: <code>_sample_ar()</code> — Acceptance-Rejection	9
4.5	<code>sample()</code> and <code>get_samples()</code>	10
4.6	<code>mc_phi(u, return_samples=False)</code>	10
4.7	<code>kernel_rff(u_list)</code>	10
4.8	<code>plot_kernel_comparison(t_max, n_points, direction, save_path)</code>	10
4.9	<code>save_kernel_comparison(path, t_max, n_points=100)</code>	10
5	Class SVE_Simulator	11
5.1	Constructor	11
5.2	<code>simulate(algo, M=100, brownian_path=None)</code>	11
5.3	<code>compare_paths(M=100, same_brownian=True, save_path=None)</code>	11
5.4	<code>benchmark_elapsed_time(M, N_t_values, n_repeats=1, save_path=None)</code>	11
5.5	<code>lp_strong_error(M_values, p_values, N_MC=30, same_brownian=False, save_path=None)</code>	11
5.6	<code>weak_phi_error(M_values, phi_functions, N_MC=30, same_brownian=False, error_type='weak', save_path=None)</code>	12
6	Utilities: RFFVolterraSfBM_Uutils.py	13
7	Default Parameters and Import Conventions	13

1 Mathematical Background

1.1 The SfBM kernel

The Stationary fractional Brownian Motion driving process has kernel

$$K(\tau) = \frac{\nu^2}{2} \left(1 - (\|\tau\|/T)^{2H} \right) \mathbf{1}_{\|\tau\| \leq T}, \quad (1)$$

where $\nu^2 > 0$ is the squared amplitude, $H \in (0, \frac{1}{2})$ is the Hurst exponent, and $T > 0$ is the integral (decorrelation) scale. This is the same kernel used throughout the LogSfBM project.

1.2 Spectral density f_ω and the Bochner identity

The Bochner theorem guarantees that K is positive-definite and that there exists a *spectral density* $f_\omega \geq 0$ such that

$$K(\tau) = \int_{\mathbb{R}^d} e^{i\xi^\top \tau} f_\omega(\xi) d\xi \quad (\text{Fourier convention: unitary, angular frequency}). \quad (2)$$

For the SfBM kernel (1), the spectral density admits the **closed-form** expression using hypergeometric functions.

Closed-form expression (factored representation).

$$f_\omega(x) = \frac{\nu^2 T^d}{\frac{2}{\pi^{d/2}} \left(\frac{d}{2H} + 1 \right) \Gamma(\frac{d}{2} + 1)} \cdot {}_1F_2\left(\frac{d}{2} + H; \frac{d}{2} + H + 1, \frac{d}{2} + 1; -\frac{\|x\|^2 T^2}{4}\right), \quad (3)$$

where ${}_1F_2$ is the generalised hypergeometric function evaluated at high precision via `mpmath`.

Numerical formula (radial Hankel integral). An equivalent representation integrates the kernel directly. For $d = 1$:

$$f_\omega^{(d=1)}(x) = \frac{2}{(2\pi)^{1/2}} \int_0^T \frac{\nu^2}{2} \left(1 - \frac{u^{2H}}{T^{2H}} \right) \cos(xu) du, \quad (4)$$

computed via `scipy.integrate.quad` with a cosine weight. For $d \geq 2$ the radial reduction gives

$$f_\omega^{(d \geq 2)}(x) = \frac{\nu^2/2}{\|x\|^{d/2+1}} \int_0^{\|x\|T} \left(1 - \frac{u^{2H}}{(\|x\|T)^{2H}} \right) u^{d/2} J_{d/2-1}(u) du, \quad (5)$$

computed via adaptive Simpson integration with automatic refinement up to $32\times$ oversampling.

Note

Two normalisation conventions. The numeric formula (4)–(5) and the closed-form (3) use the same FT convention but differ by a constant prefactor arising from the two

algebraic forms of the hypergeometric representation used in the original script. Both are valid spectral densities for the same kernel; the closed-form is faster and more accurate; the numeric integral provides an independent sanity check.

1.3 Random Fourier Features (RFF)

The fundamental identity underlying RFF is:

$$K(\tau) = \frac{\nu^2}{2} \mathbb{E}_{\eta \sim f_\omega} \left[\cos(\eta^\top \tau) \right]. \quad (6)$$

Drawing M i.i.d. samples $\eta_1, \dots, \eta_M \sim f_\omega$ gives the unbiased Monte-Carlo approximation

$$K^M(\tau) = \frac{\nu^2}{2M} \sum_{m=1}^M \cos(\eta_m^\top \tau) \xrightarrow{M \rightarrow \infty} K(\tau). \quad (7)$$

1.4 Stochastic Volterra Equation (SVE)

The SVE considered is

$$X_t = x_0 + \int_0^t K(t-s) \sigma(s, X_s) dW_s, \quad t \in [0, T_f], \quad (8)$$

where W_t is a standard Brownian motion and $\sigma : [0, T_f] \times \mathbb{R} \rightarrow \mathbb{R}$ is the volatility function. The time grid is $t_n = n T_f / N$, $n = 0, \dots, N$, with uniform step $\Delta t = T_f / N$.

1.4.1 Algorithm 1: Euler scheme ($\mathcal{O}(N^2)$)

Algorithm

1. Set $X_0 = x_0$. Draw $G_n \sim \mathcal{N}(0, 1)$, $n = 0, \dots, N-1$.
2. For $n = 0, \dots, N-1$:

$$X_{n+1} = x_0 + \sum_{i=0}^{n-1} K(t_n - t_i) \sigma(t_i, X_i) \sqrt{\Delta t} G_i.$$

Cost: $\mathcal{O}(N^2)$ kernel evaluations per path.

1.4.2 Algorithm 2: Euler with RFF ($\mathcal{O}(NM)$)

Replace K by K^M and maintain cosine/sine running integrals $I_c^{(m)}, I_s^{(m)}$ updated at each step.

Algorithm

1. Draw $\eta_1, \dots, \eta_M \sim f_\omega$ (via `SpectralSampler`).
2. Precompute $\cos(\eta_m t_n)$ and $\sin(\eta_m t_n)$ for all m, n : arrays of shape $(M, N+1)$.
3. Initialise $X_0 = x_0$, $\mathbf{I}_c = \mathbf{I}_s = \mathbf{0} \in \mathbb{R}^M$.

4. For $n = 0, \dots, N - 1$:

$$\begin{aligned}\mathbf{I}_c &+= \cos(\eta \cdot t_n) \sigma(t_n, X_n) \sqrt{\Delta t} G_n, \\ \mathbf{I}_s &+= \sin(\eta \cdot t_n) \sigma(t_n, X_n) \sqrt{\Delta t} G_n, \\ X_{n+1} &= x_0 + \frac{K(0)}{M} \left(\cos(\eta \cdot t_{n+1})^\top \mathbf{I}_c + \sin(\eta \cdot t_{n+1})^\top \mathbf{I}_s \right).\end{aligned}$$

Cost: $\mathcal{O}(NM)$ operations per path, with a one-time $\mathcal{O}(NM)$ precomputation of the trig arrays. This is quasi-linear in N once $M \ll N$.

2 Module Architecture

The two files provide complementary layers:

File	Role
RFFVolterraSfBM_Class.py	The three main classes: SpectralDensity , SpectralSampler , SVE_Simulator . Also exposes the module-level helper _generate_grid .
RFFVolterraSfBM_Utills.py	Standalone utility functions preserved for backward-compatibility with existing call sites. Includes the lambda-to-L ^A T _E X mapper, grid helper, numeric/closed-form FT functions, standalone mc_phi , kernel_reconstruction_RFF , sample_from_density , and the Haar-wavelet pre-estimator EstH_RFF .

Class / function	Summary
SpectralDensity (nu2,H,T,d) spectral_density_numeric (x) spectral_density_closedform (x) compare (xmin,xmax,npts) save_comparison (path,...) inverse_fourier_transform (x)	Spectral density evaluation and comparison. Numerical radial integral (Eqs. 4–5). Hypergeometric closed form (Eq. 3). Grid comparison, returns (pts , Fnum , Frhs , abs_err). Saves comparison to Excel (.xlsx). Inversion check: recovers $K(x)$ from f_ω numerically.
SpectralSampler (nu2,H,T,d,n_samples,method) sample () mc_phi (u, return_samples) kernel_rff (u_list) plot_kernel_comparison (...) save_kernel_comparison (path,...)	Sample from f_ω and reconstruct K^M . Draw M frequencies; cache in self._samples . $\hat{\mathbb{E}}[\cos(\eta^\top u)]$ (Eq. 6). $K^M(u) = \frac{\nu^2}{2} \cdot \text{mc_phi}(u)$ (Eq. 7). Plot K^M vs K and save figure. Saves comparison to Excel (.xlsx).
SVE_Simulator (H,T,nu,sigma,x0,T_final,N_t) simulate (algo,M,brownian_path) compare_paths (M,same_brownian,save_path) benchmark_elapsed_time (M,N_t_values,...) lp_strong_error (M_values,p_values,N_MC,...) weak_phi_error (M_values,phi_functions,...)	Simulate SVE (8) and measure convergence. Single-path simulation: 'Euler' or 'Euler with RFF'. Side-by-side plot of both algorithms. Timing table vs N . $\mathbb{E}[\sup_i X_i - X_i^M ^p]$ vs M with Monte-Carlo error bars. $ \mathbb{E}[\phi(X_{T_i})] - \mathbb{E}[\phi(X_{T_i}^M)] $ vs M .

3 Class SpectralDensity

3.1 Constructor: `__init__(nu2, H, T, d=1)`

Parameter	Type	Description
nu2	float	Squared amplitude $\nu^2 > 0$.
H	float	Hurst exponent $H \in (0, \frac{1}{2})$.
T	float	Integral scale $T > 0$.
d	int	Dimension (default 1).

3.2 `spectral_density_numeric(x, method='simpson')`

Computes $f_\omega(x)$ via numerical quadrature.

$d = 1$: Calls `scipy.integrate.quad` with `weight='cos'` on the integrand $(\nu^2/2)(1-(u/T)^{2H})$ over $[0, T]$:

$$f_\omega^{(d=1)}(x) = \frac{2}{(2\pi)^{1/2}} \int_0^T \frac{\nu^2}{2} \left(1 - \frac{u^{2H}}{T^{2H}}\right) \cos(xu) du.$$

$d \geq 2$: Uses adaptive Simpson on the Hankel integrand $h(u) = (1 - (u/U)^{2H}) u^{d/2} J_{d/2-1}(u)$ over $[0, U]$ with $U = k \cdot T$ and $k = \|x\|$. The integration grid doubles progressively (factor $\times 2$ up to $\times 32$) until the relative change falls below 10^{-6} or the absolute change below 10^{-12} . Special limit at $k = 0$: $f_\omega^{(d=2)}(0) = (\nu^2/2) \cdot H/(2(1+H)) \cdot T^2$.

3.3 `spectral_density_closedform(x)`

Evaluates the closed-form (3) at a scalar or d -dimensional point x :

$$f_\omega(x) = \underbrace{\frac{\nu^2 T^d}{\frac{2}{\pi^{d/2}} \left(\frac{d}{2H} + 1\right) \Gamma\left(\frac{d}{2} + 1\right)}}_{\text{prefactor}} \cdot {}_1F_2\left(\frac{d}{2} + H; \frac{d}{2} + H + 1, \frac{d}{2} + 1; -\frac{\|x\|^2 T^2}{4}\right),$$

using `mpmath` with 50 decimal digits of precision.

3.4 `compare(xmin, xmax, npts)`

Builds a d -dimensional grid with `npts` points per axis in `[xmin, xmax]d` (always including 0), evaluates both `spectral_density_numeric` and `spectral_density_closedform` at every grid point, and returns `(pts, Fnum, Frhs, abs_err)`, all of shape `(nptsd, ...)`. Prints **Max abs error** to stdout.

3.5 `save_comparison(path, xmin=1.0, xmax=8.0, npts=30)`

Calls `compare`, computes the relative error

$$\varepsilon_{\text{rel}}(x) = \frac{|F_{\text{num}}(x) - F_{\text{rhs}}(x)|}{\max(|F_{\text{num}}(x)|, |F_{\text{rhs}}(x)|, 10^{-15})},$$

and writes a DataFrame with columns `pt`, `Fnum`, `Frhs`, `abs_err`, `rel_err` to the Excel file at `path` (parent directory created automatically). Returns the path string.

3.6 `inverse_fourier_transform(x)`

Computes the inverse FT of f_ω numerically as a consistency check. For $d = 1$ integrates $f_\omega(u) \cos(xu)$ over $[-100T, 100T]$ via `scipy.integrate.quad`. For $d \geq 2$ applies the radial Hankel inversion:

$$K(x) = \rho^{-(d/2-1)} \int_0^{100T} r^{d/2} f_\omega(r) J_{d/2-1}(\rho r) dr, \quad \rho = \|x\|.$$

Should return values close to $K(x)$ (Eq. 1).

4 Class SpectralSampler

4.1 Constructor: `__init__(nu2, H, T, d=1, n_samples=1000, method='Hamiltonian MC')`

Parameter	Type	Description
<code>nu2</code>	float	ν^2 .
<code>H</code>	float	Hurst exponent.
<code>T</code>	float	Integral scale.
<code>d</code>	int	Dimension (1 or 2 for AR; any for HMC).
<code>n_samples</code>	int	M : number of spectral frequencies to draw.
<code>method</code>	str	'Hamiltonian MC' or 'Acceptance Rejection'.

Samples are cached in `self._samples` after the first call to `sample`; subsequent calls to `mc_phi` and `kernel_rff` reuse them via `get_samples`.

4.2 Radial density `_radial_pdf(r)`

The unnormalised radial spectral density

$$p_r(r) \propto \frac{f_\omega(r e_1)}{\nu^2/2} = \frac{1}{\nu^2/2} \cdot \frac{(T^2/2)^{d/2} \nu^2}{\Gamma(d/2)} \left[\frac{1}{d} {}_0F_1\left(\frac{d}{2} + 1; -\frac{T^2 r^2}{4}\right) - \frac{1}{d + 2H} {}_1F_2\left(\frac{d}{2} + H; \frac{d}{2}, \frac{d}{2} + H + 1; -\frac{T^2 r^2}{4}\right) \right],$$

clipped to zero if numerically negative. Evaluated via `mpmath` at 50 dps.

4.3 Sampling backend 1: `_sample_hmc()` — Hamiltonian Monte Carlo

Implements a standard HMC sampler for the log-density $\log p_r(\|\cdot\|)$ with finite-difference gradients (step $\varepsilon = 10^{-6}$).

Algorithm

HMC parameters: step size $\epsilon = 0.05$, $L = 20$ leapfrog steps.

1. Initialise $x = \mathbf{0}_d$.
2. For each of M samples:
 - (a) Draw momentum $p \sim \mathcal{N}(0, I_d)$; save $p_0 = p$.
 - (b) Half-step: $p \leftarrow p + \frac{\epsilon}{2} \nabla \log p_r(x)$.
 - (c) $L - 1$ full leapfrog steps: $x \leftarrow x + \epsilon p$, $p \leftarrow p + \epsilon \nabla \log p_r(x)$.
 - (d) Final half-step: $p \leftarrow p + \frac{\epsilon}{2} \nabla \log p_r(x)$.
 - (e) Metropolis–Hastings acceptance with log-ratio $\log \alpha = \log p_r(x_{\text{prop}}) - \log p_r(x) - \frac{1}{2} \|p\|^2 + \frac{1}{2} \|p_0\|^2$.
3. Return samples array of shape (M, d) .

Works for any dimension d . No proposal tuning required.

4.4 Sampling backend 2: `_sample_ar()` — Acceptance-Rejection

Uses an F -distribution proposal for the *radial* component only (valid for $d \in \{1, 2\}$; raises `ValueError` otherwise).

Algorithm

Proposal: $q(r) = 0.45 \times 100 \times f_F(r; 2.25, 0.25, 100)$ where f_F is the F -distribution pdf with parameters $(\nu_1, \nu_2, \text{scale}) = (2.25, 0.25, 100)$.

1. Repeat until M radii $R_1, \dots, R_M$ accepted: draw $R_f \sim f_F(\cdot; 2.25, 0.25, 100)$; accept if $U \sim \mathcal{U}[0, 1] < p_r(R_f)/q(R_f)$.
2. For $d = 1$: return R as shape $(M, 1)$.
3. For $d = 2$: multiply each radius by a uniform unit direction, $\eta_m = R_m \cdot Z_m / \|Z_m\|$ with $Z_m \sim \mathcal{N}(0, I_2)$.

4.5 sample() and get_samples()

`sample` dispatches to either backend and stores the result in `self._samples` (shape (M, d)). `get_samples` returns `self._samples`, calling `sample` lazily if it has not been called yet.

4.6 mc_phi(u, return_samples=False)

Monte-Carlo estimate of the characteristic function:

$$\hat{\phi}(u) = \frac{1}{M} \sum_{m=1}^M \cos(\eta_m^\top u) \approx \mathbb{E}_{\eta \sim f_\omega} [\cos(\eta^\top u)].$$

With `return_samples=True` returns the per-sample cosine values as an array of shape $(1, M)$ (needed by the GMM error-vector builders that require an unsqueezed row for Newey–West estimation).

Note

$\hat{\phi}(0) = 1$ exactly, for any sample set (since $\cos(0) = 1$). This is verified by unit test `test_mc_phi_at_zero_is_one`.

4.7 kernel_rff(u_list)

Evaluates Eq. (7) at an array of evaluation points:

$$K^M(u) = \frac{\nu^2}{2} \hat{\phi}(u), \quad \forall u \in \mathbf{u_list}.$$

Accepts `u_list` of shape (n_{pts}, d) or $(n_{\text{pts}},)$ and returns shape $(n_{\text{pts}},)$.

4.8 plot_kernel_comparison(t_max, n_points, direction, save_path)

Evaluates K^M and the exact K on a grid $[-t_{\text{max}}, t_{\text{max}}]^d$ and plots them:

- $d = 1$: two-panel figure (top: K^M vs K ; bottom: semi-log absolute error).
- $d = 2$: three-panel 3D surface figure (RFF, exact, error) using the `cividis` colormap.

Saves to `save_path` if provided.

4.9 save_kernel_comparison(path, t_max, n_points=100)

Writes an Excel file with columns `pt`, `RFF`, `ClosedForm`, `abs_err`, `rel_err` containing the comparison of K^M vs K on the grid $[-t_{\text{max}}, t_{\text{max}}]^d$.

5 Class SVE_Simulator

5.1 Constructor

Parameter	Type	Description
H	float	Hurst exponent.
T	float	Integral scale of the SfBM kernel.
nu	float	$\nu = \sqrt{\nu^2}$ (amplitude).
sigma	callable	Volatility: $\sigma(t, x) \rightarrow \mathbb{R}$.
x0	float	Initial condition $X_0 = x_0$ (default 0).
T_final	float	Simulation horizon T_f (default 1).
N_t	int	Number of time steps N (default 1000).

The exact SfBM kernel (1) is stored as the closure `self._K`.

5.2 `simulate(algo, M=100, brownian_path=None)`

Generates one path $X_0, \dots, X_N$ of the SVE (8).

algo	Description
'Euler'	$\mathcal{O}(N^2)$ brute-force scheme (<code>_euler_slow</code>). Uses the exact kernel K at each lag.
'Euler with RFF'	$\mathcal{O}(NM)$ fast scheme (<code>_euler_rff</code>). Draws M spectral frequencies from <code>SpectralSampler</code> (HMC, $d = 1$) and accumulates cosine/sine running integrals.

`brownian_path` is an optional array of N pre-drawn $\mathcal{N}(0, 1)$ increments shared across schemes for variance reduction. Returns `X`: `ndarray`, shape $(N + 1,)$, with `X[0] = x0`.

5.3 `compare_paths(M=100, same_brownian=True, save_path=None)`

Draws one Brownian path, simulates both algorithms, plots the two trajectories X (Euler) and X^M (Euler with RFF) on the same axes, and optionally saves the figure. Returns `(X_euler, X_rff)`.

5.4 `benchmark_elapsed_time(M, N_t_values, n_repeats=1, save_path=None)`

Measures the wall-clock time of both algorithms for each $N \in \text{N_t_values}$, averaging over `n_repeats` independent runs. Returns a `pd.DataFrame` with columns `N_t`, `Elapsed_time_Euler`, `Elapsed_time_RFF` and optionally saves to Excel.

5.5 `lp_strong_error(M_values, p_values, N_MC=30, same_brownian=False, save_path=None)`

Estimates the strong L^p error between Algorithm 1 and Algorithm 2 as a function of M :

$$\varepsilon_p(M) = \mathbb{E} \left[\sup_{0 \leq n \leq N} |X_n - X_n^M|^p \right]^{1/p} \approx \left(\frac{1}{N_{\text{MC}}} \sum_{r=1}^{N_{\text{MC}}} \left(\max_n |X_n^{(r)} - X_n^{M,(r)}| \right)^p \right)^{1/p}. \quad (9)$$

For each $M \in \text{M_values}$ and $p \in \text{p_values}$, N_{MC} independent path pairs are simulated (optionally sharing the same Brownian increments). Results are returned as dicts `results_mean[p]`, `results_std[p]`, and an error-bar figure and Excel file are saved if `save_path` is provided.

5.6 `weak_phi_error(M_values, phi_functions, N_MC=30, same_brownian=False, error_type='weak', save_path=None)`

Estimates the weak or strong observable error at the terminal time T_f :

$$\text{weak: } \varepsilon_{\phi, M}^w = |\mathbb{E}[\phi(X_{T_f})] - \mathbb{E}[\phi(X_{T_f}^M)]|, \quad (10)$$

$$\text{strong: } \varepsilon_{\phi, M}^s = \mathbb{E}[|\phi(X_{T_f}) - \phi(X_{T_f}^M)|]. \quad (11)$$

`phi_functions` is a list of callables $\phi : \mathbb{R} \rightarrow \mathbb{R}$. Returns `error_vals`: ndarray, shape $(\text{len}(\text{phi_functions}), \text{len}(\text{M_values}))$. Legend labels are generated via `lambda_to_latex_mapping` from `RFFVolterraSfBM_Uutils.py`.

6 Utilities: RFFVolterraSfBM_Utills.py

Every standalone function from the original script is preserved as a module-level function. Internally, `sample_from_density` is a thin wrapper that instantiates a `SpectralSampler`; all others are self-contained.

Function	Description
<code>lambda_to_latex_mapping(phi, arg)</code>	Parses the AST of a Python <code>lambda</code> and converts it to a L ^A T _E X label string $\phi : x \mapsto \dots$. Handles <code>Pow</code> , <code>Mult</code> , <code>Div</code> , <code>Add</code> , <code>Sub</code> , <code>USub</code> , and calls to <code>sin/cos/tan/log/exp/sqrt</code> . Falls back gracefully.
<code>generate_grid(d, xmin, xmax, npts)</code>	Same as <code>_generate_grid</code> but exposed as a public function. Builds a d -dimensional grid in $[\text{xmin}, \text{xmax}]^d$ with 0 always included; returns <code>(pts, axes)</code> .
<code>FourierTransform_numeric(x, alpha, d)</code>	Numerical FT of $(1 - \ u\ ^\alpha) \mathbf{1}_{\ u\ \leq 1}$ via radial Bessel reduction.
<code>FourierTransform_fromCloseForm(x, alpha, d)</code>	Closed-form FT of same function via ${}_0F_1$ and ${}_1F_2$.
<code>FourierTransformSfBMkernel_numeric(nu, H, T, d)</code>	Numerical FT of the SfBM kernel K : mirrors <code>SpectralDensity.spectral_density_numeric</code> .
<code>FourierTransformSfBMkernel_fromCloseForm(nu2, H, T, d)</code>	Closed-form FT of K : mirrors <code>SpectralDensity.spectral_density_closedform</code> .
<code>radial_pdf_on_r(r, nu, T, H, d)</code>	Unnormalised radial density $p_r(r) \propto f_\omega(r e_1)/(\nu^2/2)$ via the ${}_0F_1/{}_1F_2$ representation.
<code>mc_phi(samples, u, return_samples)</code>	Standalone $\hat{\phi}(u) = \frac{1}{M} \sum_m \cos(\eta_m^\top u)$ from a pre-drawn sample array of shape (M, d) .
<code>kernel_reconstruction_RFF(samples, u_list, nu2)</code>	$K^M(u) = \frac{\nu^2}{2} \hat{\phi}(u)$ from pre-drawn samples.
<code>sample_from_density(n, nu, T, H, d, method)</code>	Thin wrapper: instantiates <code>SpectralSampler</code> and calls <code>sample</code> .
<code>EstH_RFF(xvals, lagSig)</code>	Haar-wavelet pre-estimator of (H, λ^2) from a log-vol series. Matches the <code>EstH</code> convention used in the GMM module.

7 Default Parameters and Import Conventions

Parameter	Default	Notes
mpmath precision	50 dps	Set at module load: <code>mp.mp.dps = 50</code> .
HMC step size	0.05	<code>step_size</code> in <code>_sample_hmc</code> .
HMC leapfrog steps	20	<code>n_leapfrog</code> in <code>_sample_hmc</code> .
HMC gradient step	10^{-6}	Finite-difference step ε .
AR proposal	$F(2.25, 0.25, 100)$	<code>p_dfn=2.25</code> , <code>p_dfd=0.25</code> , <code>p_scale=100</code> .
AR rescale	$\times 0.45 \times 100$	Ensures $q \geq p_r$ everywhere.
Simpson refinement	up to $\times 32$	For <code>spectral_density_numeric</code> $d \geq 2$.
SVE default N_t	1000	Steps in <code>SVE_Simulator</code> .
SVE default M	100	RFF features in <code>simulate</code> .
matplotlib backend	Agg	Set at import; no interactive display.

Import conventions.

```
from RFFVolterraSfBM_Class import (  
    SpectralDensity,  
    SpectralSampler,  
    SVE_Simulator,  
    _generate_grid,  
)  
from Utils import (  
    generate_grid,  
    FourierTransformSfBMkernel_numeric,  
    FourierTransformSfBMkernel_fromCloseForm,  
    mc_phi,  
    kernel_reconstruction_RFF,  
    lambda_to_latex_mapping,  
    sample_from_density,  
    EstH_RFF,  
)
```